



Higher National Diploma in Computing

(Software Engineering)

Unit 1: Programming

Development Report

North Sussex Judo – Monthly Training Fee Calculator

Academic Year: 2024/2025

Assignment Title: Introduction to Algorithms and Programming with an IDE

Name: Vihanga Shamod Hewanayake

Student ID: 521546

1. Introduction

This report details the development journey undertaken to create my initial project at AQ Digital Solutions (AQDS) called North Sussex Judo Monthly Training Fees. The client was a small local Judo Club wanting a simple command-line program that will compute and report monthly training fees for its Judo athletes. The program considers the training schedule that each athlete has and any private coaching hours they attend, competitions they are involved in during the month and their current weight vs the weight limit of their competition class.

Before writing any code, I will outline the thought process of my algorithm; I will take you through the steps of turning the algorithm into an actual program, describe some problems I encountered whilst working on the program, and through some coding standards that I used throughout my work. I will also describe how the original algorithm was improved, enhanced and developed along the way. The actual program consists of 296 lines in a single Python file and was written in six Git commits corresponding to different stages of its development.

2. Defining the Algorithm

2.1 Problem Decomposition

I read the brief thoroughly and conceived the basic processes of the problem before writing a single line of code. An algorithm is a finite, ordered sequence of unambiguous steps to solve a particular problem (Aho, Hopcroft and Ullman, 1987). I needed to find out exactly what those instructions had to be to work in this project.

I found six major operations that the program needed to perform:

- Collect and verify athlete data from the user: including name, training plan, current weight, competition category, number of competitions entered and private coaching hours.
- Work out monthly training plan fee: All prices x 4 weeks/month - Beginner £25.00/week, Intermediate £30.00/week, Elite £35.00/week.
- Private coaching fees £9.50 per hour per week (maximum of 5 hours per week)
- Calculate the cost of competition entry at £22.00 per competition (Elite and Intermediate athletes only). Beginners cannot compete.
- A comparison of the athlete's current weight with the maximum for his/her registered competition weight class.
- Print out a detailed list of all expenses by month and provide a grand total, both to 2 decimal places.

2.2 Pseudocode Algorithm

I made decision to use pseudocode for the main design tool because it is language independent and closely resembles real code, but it isn't language specific. Before any coding started, I had designed a complete algorithm that I copied into the following pseudocode.

```
ALGORITHM: North Sussex Judo Fee Calculator
BEGIN
    SET athletes = empty list
    LOOP (main menu event loop):
        DISPLAY menu: Register / View Report / Summary / Exit
        READ choice
```

```

IF choice = 1 (Register) THEN
    READ name                -- not blank, must be unique
    DISPLAY plan options (Beginner / Intermediate / Elite)
    READ plan                 -- must be 1, 2 or 3
    READ weight               -- positive decimal number
    DISPLAY weight category options (Flyweight to Heavyweight)
    READ category             -- must be 1 to 6
    IF plan = Beginner THEN
        SET competitions = 0  -- Beginners cannot enter competitions
    ELSE
        READ competitions    -- whole number 0 or above
    END IF
    READ wants_coaching       -- y or n
    IF wants_coaching = yes THEN
        READ coaching_hours  -- 0.0 to 5.0 hours per week
    END IF
    CREATE Athlete with all collected values
    APPEND Athlete to athletes
ELSE IF choice = 2 (View Report) THEN
    DISPLAY numbered athlete list
    READ selection
    athlete = athletes[selection]
    training_fee    = athlete.weekly_rate * 4
    coaching_fee    = min(athlete.coaching, 5) * 4 * 9.50
    IF athlete.plan = Intermediate OR Elite THEN
        competition_fee = athlete.competitions * 22.00
    ELSE
        competition_fee = 0.00
    END IF
    total = training_fee + coaching_fee + competition_fee
    DISPLAY name, plan, weight status, itemised costs, total
ELSE IF choice = 3 (Summary) THEN
    FOR EACH athlete IN athletes
        CALCULATE total fee, DISPLAY name, plan, total
    END FOR
    DISPLAY grand total
ELSE IF choice = 4 (Exit) THEN
    READ confirm -- IF yes THEN END program
END IF
END LOOP
END

```

2.3 Flowchart

I created a flowchart to illustrate the decision points and loops along with the pseudocode I was working on. Submitting this flowchart with pseudocode before coding helped me to notice the Heavyweight None and the coaching cap.

2.4 Why Pseudocode and Flowcharts

Pseudocode is great for testifying to the 'logical flow' and being used for communication with non-programmers. Branching is visible in the flowchart. The two of them together gave me the confidence that I'd have done the right thing up front and saved myself the time of checking out the syntax of Python. Designing before coding always yields a better results as stated by Reeves (1992).

3. Steps to Build the Application

3.1 Why Python

I chose python3, as it's syntax was very similar that of the natural language of the programmer, thereby making it easy to translate the pseudocode. All three paradigms are supported in one file natively with Python. It is a language that runs without compile, and it's the main language at AQDS.

3.2 Code Structure

The program is compact, storing just 296 lines of code in one file, with each section having one main job and is labelled.

Section	Paradigm	Contents
Constants block	N/A	PLANS, CATEGORIES, COACHING_RATE, COMP_FEE, WEEKS, MAX_COACHING
Athlete class	Object-Oriented	__init__, can_compete(), weight_status(), __str__()
calc_* functions	Procedural	calc_training(), calc_coaching(), calc_competitions(), calc_total()
ask_* functions	Procedural	ask_string(), ask_number(), ask_yes_no()
choose_* functions	Procedural	choose_plan(), choose_category(), clear_screen()
print_* functions	Procedural	print_header(), print_report(), print_summary()
handle_* functions	Event-Driven	handle_register(), handle_report(), handle_summary(), handle_exit()
main()	Event-Driven	Dispatch table + while True event loop

3.3 Build Order

I built the code in stages. One commit per stage. So each component could be tested before it goes to the next depended.

1. Initial commit-README (3689caf).

2. Added constants and Athlete classes (75eb17f) – PLANS, CATEGORIES, COACHING_RATE (£9.50), COMP_FEE (£22.00), WEEKS (4), MAX_COACHING (5) and the Athlete class with `can_compete()` and `weight_status()`.
3. Added calculation functions (c9fec1f) – `calc_training()`, `calc_coaching()`, `calc_competitions()` and `calc_total()`. Verified: Beginner £25 × 4 = £100.00; Elite with 3hr coaching + 1 comp = £276.00; max 5hr coaching = £190.00.
4. Added input validations (3a64eca) – `ask_string()`, `ask_number()`, `ask_yes_no()`, `choose_plan()` and `choose_category()`. Each loops until valid input received.
5. Added Display functions (d7b0bea) – `print_header()`, `print_report()` and `print_summary()` kept separate from calculation logic.
6. Added Event handlers and main loop (124d0f3) – `handle_register()`, `handle_report()`, `handle_summary()`, `handle_exit()` and `main()` with dispatch table. Program complete.

4. From Algorithm to Working Code

4.1 What Stayed the Same

The main LOOP became `while True` in `main()`. The four IF branches became four handler functions in the dispatch table. The steps of input were replaced with the following calls in the same order as input steps: `ask_string`, `ask_number`, `choose_plan`, `choose_category`. The fee formulas were changed to `calc_training()`, `calc_coaching()` and `calc_competitions()`, while the arithmetic is the same.

4.2 What Changed and Why

The IF/ELSE menu chain was replaced with a dictionary dispatch table as a cleaner and more Pythonic and easier to add to. The loosely defined Athlete object has been updated to make it a proper class that has methods `can_compete()` and `weight_status()`. In order to match the brief, the `:.2f` notation was used on all f-strings to display the results to 2 decimal places.

4.3 The Relationship Between Algorithm and Code

The relation is highly concrete. Nearly all of the pseudocode blocks correspond to a named function. The big gap is input validation, pseudocode assumes clean input whereas an actual code would have to have an `ask_string()`, `ask_number()`, `ask_yes_no()`, `choose_plan()`, `choose_category()`, etc. to deal with all the ways in which users can enter invalid data.

5. Challenges Faced During Development

5.1 Handling Bad User Inputs

The `input()` function in Python has always been a string, meaning that there was a need for conversion and error checking everywhere. To use both integers and decimals in one function, I added `ask_number` with the `whole=True/False` as a parameter and an `except ValueError` to catch the whole and the decimal numbers and return the result in both cases under the same function name.

5.2 Heavyweight Category and None

There is no maximum weight allowed on Heavyweight category. So I tried to put numeric ceiling on all my categories which would cause `weight_status()` to raise a `TypeError` in Runtime. I fixed it by storing `None` as the Heavyweight limit and added a `None` guard at the top of `weight_status()` before any arithmetic was to be done. So the issue fixed.

5.3 Beginner Competition Eligibility

There is a competition that's open just for the Intermediate and Elite. I placed the check inside `can_compete()` on the `Athlete` class, such that the object of the class contains the rule, and every time the check is needed, it gets used directly by `calc_competitions()`, barring any 'bypass'.

5.4 Coaching Hours Cap

Up to 5 hours per week at £9.50/Per hour. I checked this in `ask_number()` against `max=5.0` and I added the line `min(athlete.coaching, MAX_COACHING)` in `calc_coaching()` as an extra security.

6. Coding Standards Applied

In AQDS, we follows PEP 8 and standards which I followed as follows:

Standard	How Applied
Descriptive names	<code>calc_training()</code> not <code>ct()</code> , <code>handle_register()</code> not <code>reg()</code>
snake_case	<code>weekly_rate</code> , <code>cat_limit</code> , <code>ask_number</code> , <code>choose_plan</code>
UPPER_CASE constants	<code>COACHING_RATE</code> , <code>COMP_FEE</code> , <code>WEEKS</code> , <code>MAX_COACHING</code> , <code>SEPARATOR</code>
Docstrings	<code>Athlete</code> class has a docstring; functions have inline comments
Single responsibility	Each function does one thing: <code>calc_coaching()</code> only calculates coaching
~79 char line limit	Long lines split inside brackets

7. How the Algorithm Was Enhanced During Development

7.1 Fixing the Heavyweight Crash

The VS code debugger reported an error of `cat_limit` was not a number inside the `weight_status()`. Using a `None` guard before subtraction. This took the debugger one minute fix.

7.2 Replacing if/elif with Dispatch Table

I used if/elif to route the main(), that was the way I had it before. Instead I used a dictionary of choice strings to lambda functions that are cleaner, shorter and easier to extend.

7.3 Version Control with Git

I used the practices of using Git for all execution of the exercises and to commit after every logical step. The six commits below are the same as the one present in the GitHub repository – github.com/Vihangashamod/NorthSussexJudo:

```
124d0f3 Added Event handlers and main loop
d7b0bea Added Display functions
3a64eca Added input validations
c9fec1f Added calculation functions
75eb17f Added constants and Athlete classes
3689caf Initial commit-README
```

That is, if I ever got(handle_exit) broken, I was able to do "git diff" for a few minutes in order to see what changed on a line-by-line basis and back this patch in seconds.

7.4 Defence Cap in calc_coaching()

I tested that it was important to have the min(athlete.coaching,MAX_COACHING) inside the calc_coaching,newly created Athlete object would otherwise get the maximum value, anyway.

8. Conclusion

Constructing the North Sussex Judo calculator involved me in getting hands-on experience of the entire software development lifecycle. The pseudo code and flowchart portions were also more beneficial than I thought they would be – these two bugs (the Heavyweight and coaching cap) were uncovered before I had written any code. A history of all the decisions made as they evolved over 6 commits can be seen at github.com/Vihangashamod/NorthSussexJudo. By breaking the code up into eight well labeled segments almost every part was easily testable and modifiable without requiring any other parts. By maintaining the codebase accordingly to PEP 8 and committing each step in the right direction, the development history becomes clear and all AQDS developers can be at ease to maintain the codebase.

References

Aho, A.V., Hopcroft, J.E. and Ullman, J.D. (1987) Data Structures and Algorithms. 1st edn. Reading, MA: Addison-Wesley.

Edwards, S., Shaffer, C.A. and Kapur, A. (2012) Problem Solving and Algorithms. Available at: <http://sofia.cs.vt.edu/cs1114-ebooklet/chapter4.html> (Accessed: 10 June 2024).

McConnell, S. (2004) Code Complete. 2nd edn. Redmond, WA: Microsoft Press.

Mitchell, J. (2002) Concepts in Programming Languages. Cambridge: Cambridge University Press.

Popic, S. (2018) 'The Benefits of Coding Standards Enforcement', 26th Telecommunications Forum TELFOR 2018. doi:10.1109/TELFOR.2018.8612149.

Python Software Foundation (2023) PEP 8. Available at: <https://peps.python.org/pep-0008/> (Accessed: 10 June 2024).

Reeves, J. (1992) 'What is Software Design?', C++ Journal. Available at: <http://www-public.telecom-sudparis.eu/~gibson/Teaching/Teaching-ReadingMaterial/Reeves92.pdf> (Accessed: 10 June 2024).